

Experiment No. 3

AIM

To design an Employee–Department–Project database, populate it with at least 30 employees across 5 departments and 8 projects, and demonstrate selection, projection, aggregate functions, GROUP BY, HAVING, CASE expressions, and ORDER BY using SQL queries.

METHODOLOGY

1. Design the relations by first identifying entities and their relationships.
2. Create Department and Project master tables before creating Employee records.
3. Use Employee as the central entity and connect it to Department through a foreign key.
4. Use a separate Employee_Project bridge table to represent the many-to-many relationship between employees and projects.
5. Populate the database with a different sample dataset of 30 employees, 5 departments and 8 projects.
6. Demonstrate filtering and column selection before moving to grouped analysis.
7. Apply aggregate functions to department and project data, then filter groups using HAVING.
8. Use CASE for salary-band classification and ORDER BY for meaningful presentation of results.

SCHEMA

Table	Important Columns	Purpose
Department	dept_id, dept_name, location	Stores department master data
Project	project_id, project_name, budget	Stores project information
Employee	emp_id, emp_name, dept_id, salary, hire_date	Stores employee records
Employee_Project	emp_id, project_id, hours_worked	Maps employees to projects

QUERY PLAN

Step	SQL task
1	Create the relational schema and constraints.
2	Insert departments, projects and employee records.
3	Insert employee-project assignments.
4	Perform selection and projection using WHERE and selected columns.
5	Calculate salary statistics with aggregate functions.
6	Group employees by department and project.
7	Filter groups using HAVING.
8	Classify employees with CASE and sort results using ORDER BY.

PROGRAM / SQL SCRIPT

Database and table creation:

```
CREATE DATABASE enterprise_db;
USE enterprise_db;

CREATE TABLE Department (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50) NOT NULL,
location VARCHAR(50)
);

CREATE TABLE Project (
project_id INT PRIMARY KEY,
project_name VARCHAR(80) NOT NULL,
budget DECIMAL(12,2)
);

CREATE TABLE Employee (
emp_id INT PRIMARY KEY,
emp_name VARCHAR(60) NOT NULL,
dept_id INT,
salary DECIMAL(10,2),
hire_date DATE,
FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

CREATE TABLE Employee_Project (
emp_id INT,
project_id INT,
hours_worked INT,
PRIMARY KEY (emp_id, project_id),
FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),
FOREIGN KEY (project_id) REFERENCES Project(project_id)
);
```

Insert departments and projects:

```
INSERT INTO Department VALUES
(1, 'Research & Development', 'Tower A'),
(2, 'Sales', 'Tower B'),
(3, 'Finance', 'Tower C'),
(4, 'Customer Support', 'Tower D'),
(5, 'Operations', 'Tower E');

INSERT INTO Project VALUES
(201, 'Smart Inventory', 1750000),
(202, 'FinTech Portal', 2400000),
(203, 'CRM Revamp', 1350000),
(204, 'Cloud Security', 2800000),
(205, 'Predictive AI', 3100000),
(206, 'Logistics Suite', 1900000),
(207, 'HR Automation', 1100000),
(208, 'Mobile Commerce', 1600000);
```

Insert alternate employee dataset – 30 records:

```
INSERT INTO Employee VALUES
(1, 'Vihaan Sharma', 1, 91000, '2022-01-14'),
(2, 'Myra Kapoor', 1, 83000, '2023-05-21'),
(3, 'Reyansh Mehta', 1, 97000, '2021-07-09'),
(4, 'Kiara Nair', 1, 76000, '2024-01-18'),
(5, 'Atharv Joshi', 1, 88000, '2022-11-27'),
(6, 'Anvi Rao', 1, 72000, '2024-06-03'),
(7, 'Dhruv Bhatia', 2, 69000, '2023-02-12'),
(8, 'Navya Iyer', 2, 75000, '2022-08-17'),
(9, 'Ayaan Sethi', 2, 62000, '2024-03-06'),
(10, 'Tara Malhotra', 2, 81000, '2021-10-24'),
(11, 'Rudra Shah', 2, 73000, '2023-09-15'),
(12, 'Ishani Arora', 2, 66000, '2024-05-02'),
(13, 'Samar Jain', 3, 94000, '2021-03-11'),
(14, 'Aarohi Sen', 3, 87000, '2022-06-29'),
(15, 'Yuvan Khanna', 3, 99000, '2020-12-18'),
(16, 'Avika Das', 3, 78000, '2023-04-07'),
(17, 'Parth Sood', 3, 85000, '2022-09-13'),
(18, 'Mahi Kapoor', 3, 71000, '2024-02-26'),
(19, 'Aadvik Roy', 4, 64000, '2023-01-31'),
(20, 'Zoya Khan', 4, 70000, '2022-07-16'),
(21, 'Laksh Verma', 4, 59000, '2024-04-12'),
(22, 'Sia Gupta', 4, 68000, '2023-11-08'),
(23, 'Rishaan Bose', 4, 76000, '2021-05-19'),
(24, 'Amaira Singh', 4, 61000, '2024-07-22'),
(25, 'Arnav Pillai', 5, 82000, '2022-02-04'),
(26, 'Vanya Dutta', 5, 74000, '2023-06-18'),
(27, 'Kiaan Yadav', 5, 93000, '2021-09-25'),
(28, 'Ira Mukherjee', 5, 67000, '2024-01-30'),
(29, 'Kabir Chawla', 5, 89000, '2022-12-05'),
(30, 'Rhea Thomas', 5, 78000, '2023-08-14');
```

QUERIES / OUTPUT

Employee–project assignments:

```
INSERT INTO Employee_Project VALUES
(1,205,145),(2,201,110),(3,205,160),(4,203,90),(5,204,125),
(6,207,80),(7,202,105),(8,208,115),(9,203,75),(10,202,135),
(11,206,100),(12,207,70),(13,204,150),(14,205,130),(15,202,165),
(16,208,95),(17,201,120),(18,206,85),(19,203,105),(20,207,90),
(21,208,65),(22,203,100),(23,206,140),(24,207,75),(25,201,130),
(26,206,95),(27,204,155),(28,208,80),(29,205,145),(30,201,105);
```

1. **Selection** – find employees hired after 2023 with salary above 75,000:

```
SELECT emp_id, emp_name, salary, hire_date
FROM Employee
WHERE hire_date >= '2023-01-01'
AND salary > 75000
ORDER BY hire_date;
```

2. **Projection** – show only names, salary and department IDs:

```
SELECT emp_name, salary, dept_id
FROM Employee
ORDER BY emp_name;
```

3. **Aggregate + GROUP BY** – salary statistics by department:

```
SELECT dept_id,
COUNT(emp_id) AS staff_count,
SUM(salary) AS payroll,
ROUND(AVG(salary),2) AS mean_salary,
MAX(salary) AS highest_salary
FROM Employee
GROUP BY dept_id
ORDER BY payroll DESC;
```

4. **HAVING** – departments with payroll above 430,000:

```
SELECT dept_id,
COUNT(*) AS staff_count,
SUM(salary) AS payroll
FROM Employee
GROUP BY dept_id
HAVING SUM(salary) > 430000
ORDER BY payroll DESC;
```

5. **CASE expression** – create three salary categories:

```
SELECT emp_name, salary,
CASE
WHEN salary >= 90000 THEN 'Executive'
WHEN salary >= 75000 THEN 'Professional'
ELSE 'Associate'
END AS salary_level
FROM Employee
ORDER BY salary DESC, emp_name;
```

SAMPLE OUTPUT

Department	Employees	Payroll	Average Salary
R&D	6	507,000	84,500.00
Sales	6	426,000	71,000.00
Finance	6	514,000	85,666.67
Customer Support	6	398,000	66,333.33
Operations	6	483,000	80,500.00

WORKING / ANALYSIS

A second level of analysis is performed on project participation. Instead of only studying department salaries, the project-assignment table is grouped to identify workload and participation across the project portfolio.

6. Project-wise workload using GROUP BY:

```
SELECT p.project_name,
COUNT(ep.emp_id) AS assigned_employees,
SUM(ep.hours_worked) AS total_hours,
ROUND(AVG(ep.hours_worked),2) AS avg_hours
FROM Project p
LEFT JOIN Employee_Project ep
ON p.project_id = ep.project_id
GROUP BY p.project_id, p.project_name
ORDER BY total_hours DESC;
```

7. Employees working more than 120 hours on their assigned project:

```
SELECT e.emp_name, p.project_name, ep.hours_worked
FROM Employee e
JOIN Employee_Project ep
ON e.emp_id = ep.emp_id
JOIN Project p
ON ep.project_id = p.project_id
WHERE ep.hours_worked > 120
ORDER BY ep.hours_worked DESC;
```

OBSERVATIONS / OUTCOME

No.	Observation
1	Created an Employee–Department–Project relational schema with four connected tables.
2	Inserted a new dataset containing 30 employees, 5 departments and 8 projects.
3	Demonstrated selection using multiple conditions and date-based filtering.
4	Used projection to display only the required attributes.
5	Calculated department salary statistics using COUNT, SUM, AVG and MAX.
6	Used HAVING to filter departments after aggregation.
7	Used CASE to classify employees into Executive, Professional and Associate salary levels.
8	Performed project-wise workload analysis using JOIN, GROUP BY and ORDER BY.

RESULT

The Employee–Department–Project database was implemented successfully with an alternate dataset. The required SQL operations including selection, projection, aggregate functions, GROUP BY, HAVING, CASE expressions and ORDER BY were demonstrated through both department-level and project-level analysis.

EXECUTION

```
mysql -u root -p
USE enterprise_db;

SELECT * FROM Department;
SELECT * FROM Employee;
SELECT * FROM Project;
```